11 — Account Deletion (Cascade + /deleteaccount) & Utility-Alert Poll Frequency (2026-10-15)

Part 1 — User-initiated account deletion

Motivation. Deleting a user from the Supabase Table Editor failed with `Key (id)=(100) is still referenced from table user_preferences` — the three FKs pointing at `users.id` were created without `ON DELETE CASCADE`. This blocked both ops cleanup (test user `123456`, whose dead chat also spammed every utility-alert poll with `Chat not found`) and any user privacy/GDPR-style self-deletion.

Changes

- `src/core/models.py` — `ondelete="CASCADE"` on `user_preferences.user_id`, `user_article_delivery.user_id`, `utility_alert_delivery.user_id` (new databases get the right DDL from day one)
- `src/storage/migrate.py` — `migrate_user_fk_on_delete_cascade()`: idempotent, Postgres-only (SQLite cannot alter FK constraints). Each user_id FK is dropped and re-added with `ON DELETE CASCADE` only when its current delete rule is not CASCADE, so existing databases start cleanly. Registered in `database.py` startup initialization.
- `src/core/user_service.py` — `delete_user_account(telegram_id)`: deletes child rows explicitly first (`utility_alert_shadow_log` by telegram_id — it has no FK — then the three FK tables) then the `users` row, in one transaction. The explicit deletes keep deletion working on SQLite, where the cascade migration is a no-op.
- `src/bot/handlers.py` + `src/bot/bot_main.py` — `/deleteaccount` command with a yes/cancel inline-confirmation; the callback runs the service helper via `asyncio.to_thread` (AGENTS.md rule #6) and edits the message with the outcome. Listed in `/help`.

Ops cleanup performed 2026-10-15 (before this change, manually in FK-safe order via the production DB): test user `123456` (users.id=100) deleted — 1 user_preferences, 103 user_article_delivery, 3 utility_alert_shadow_log rows. The `Chat not found` delivery failures stopped from the next poll cycle.

Verification

- `python -m compileall src` clean.
- After the deploy, confirm the startup log prints the three `ON DELETE CASCADE applied` lines once (migration is idempotent — silent on subsequent boots), then exercise `/deleteaccount` from a test chat.

Part 2 — Utility-alert poll frequency: 10 → 30 minutes

Change. `UTILITY_ALERT_POLL_MINUTES=30` set as a Fly secret on 2026-10-15 (owner-initiated, via Fly dashboard). Applies to the whole poll cycle — telegram, jbalb, Google News, and the X leg all now detect every 30 minutes.

Rationale. Cost/volume balance after adding the X leg (doc 10): X API requests drop from ~576 to ~192/day, cutting twitterapi.io spend from ~$3–5 to roughly ~$1–2/month and stretching the ~100K bonus credits roughly 3x further, while "alerts within 30 minutes" remains defensible for the FYP2 real-time-alerts chapter. The X leg's 4 searches keep their 6-second spacing (free-tier QPS: 1 request / 5 s).

Rollback. `flyctl secrets unset UTILITY_ALERT_POLL_MINUTES -a apanewsharitok` returns to the 10-minute default; or set any value ≥ 5 (config floor).

Reproduce

```
python -m compileall src
# local: /deleteaccount in a private chat with the bot
flyctl logs -a apanewsharitok # poll cadence + migration lines
```